Software Design Document (SDD)

Cover

Field	Value
Project	Department Course Timetable Management System
Document	Software Design Document
Version	v1
Related SRS version	v1
Date	10.11.2026
Team number	Team 03
Member 1	Tuna Eser · tuna645
Member 2	Levent Gürel · LeventG08
Repository	https://github.com/tuna645/seng271-team03-timetable

Revision History

Version	Date	Author(s)	Summary of changes
v1	10.11.2026	Tuna Eser, Levent Gürel	Initial class diagram and scheduling flowchart.

1. Introduction

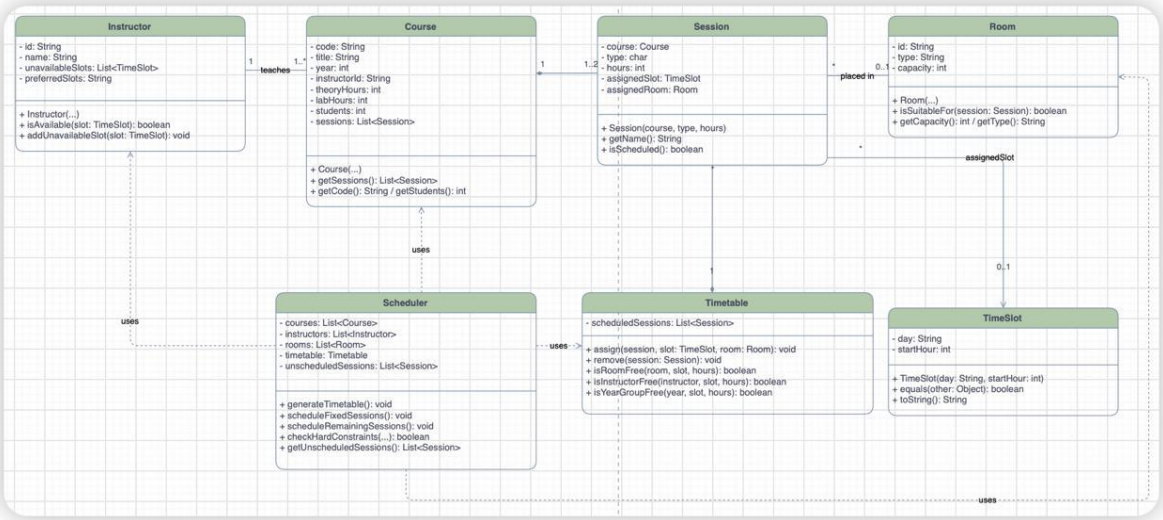
This document describes the software design and system architecture for the Department Course Timetable Management System. It is based on SRS v1 and explains how Java classes, relationships, and the scheduling algorithm work together to create weekly timetables.

2. System Overview

Part	Classes	Responsibility
User interface (menu and screens)	Main	Displays the console menu, reads user choices, and calls corresponding features.
Input (reading and validating files)	Main, Scheduler	Reads text and CSV files, parses lines into objects, and skips invalid lines with warnings.
Domain model	Course, Session, Instructor, Room, TimeSlot	Stores core data entities, attributes, and basic checks (e.g. room capacity and type).

Scheduling	Scheduler, Timetable	Runs the timetable placement algorithm, checks hard constraints, and manages slots.
Output (timetables, reports, email draft)	Timetable, Scheduler	Prints weekly grids to console, formats unscheduled reports, and creates email drafts.

3. UML Class Diagram



Tool used: draw.io · Source file: docs/design/class-diagram.drawio

☑ Every main class is shown, named exactly as in the Java code ☑ Important attributes have a type and visibility, e.g. - capacity: int ☑ Main methods have parameters and a return type, e.g. + isAvailable(slot: TimeSlot): boolean ☑ Relationships between classes are drawn, with multiplicities ☑ The class that runs the scheduling algorithm is identifiable ☑ The diagram is readable when printed on one A4 page

4. Class Descriptions

Class	Responsibility (one sentence)	Key attributes	Key methods	Related classes
Course	Stores course information and splits hours into theory and lab sessions.	code, title, year, students, sessions	getSessions(), getCode(), getStudents()	Session, Instructor
Session	Represents a single teaching block (theory or lab) to be scheduled.	type, hours, assignedSlot, assignedRoom	getName(), isScheduled(), getHours()	Course, Room, TimeSlot
Instructor	Represents an instructor and checks availability during time slots.	id, name, unavailableSlots, preferredSlots	isAvailable(slot), addUnavailableSlot(slot)	Course, TimeSlot
Room	Represents a classroom or lab and checks capacity and	id, type, capacity	isSuitableFor(session), getCapacity()	Session, Timetable

	type suitability.			
TimeSlot	Represents a one-hour scheduling unit with a day and a start hour.	day, startHour	toString(), equals(obj)	Session, Timetable
Timetable	Holds all scheduled sessions and checks whether rooms or slots are free.	scheduledSessions	assign(...), remove(...), isRoomFree(...)	Session, Room, TimeSlot
Scheduler	Controls the automatic scheduling algorithm and collects unscheduled sessions.	courses, instructors, rooms, timetable	generateTimetable(), getUnscheduledSessions()	All domain classes

5. Design Decisions

Question	Our decision	Why

How is a course split into sessions?	Each Course object creates one theory Session(-T) and/or one lab Session (-L).	Theory and laboratory classes have different room requirements and hours, so separating them makes scheduling much easier.
How is a time slot represented?	As a TimeSlot class holding a day (String) and startHour (integer, 9 to 16).	A simple day and hour pair allows quick equality checks and loops through the 40 weekly slots.
How is the timetable stored?	As an ArrayList<Session> inside the Timetableclass.	Each placed session knows its room and time slot, which makes adding, removing, and printing by year group straightforward.
How are instructor and room availability stored?	Instructor holds a list of unavailable TimeSlotobjects. Room availability is tracked by looking at current assignments in Timetable.	Simple list lookups prevent scheduling during blocked hours.
How are special requests represented?	Stored with a type (FIXED_TIME, SPECIFIC_ROOM), course code, and priority (MANDATORY or PREFERRED).	Allows fixed-time courses to be prioritized before the main algorithm runs.
How are hard constraints checked?	In dedicated methods inside Timetable and Scheduler before any assignment is saved.	Centralizing checks ensures consistent validation for both automatic scheduling and manual changes.

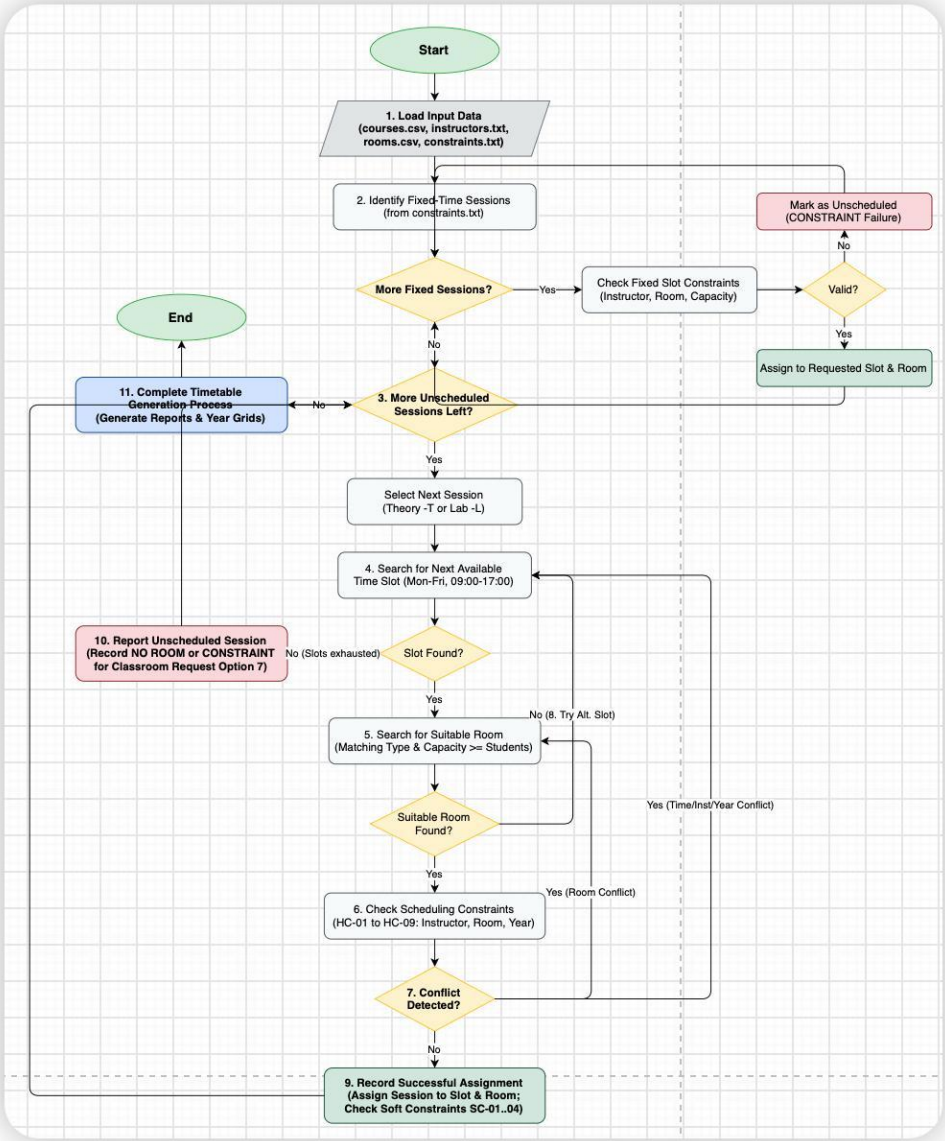
How are input errors detected and reported?	Using try-catch blocks and validation checks while reading lines in Main/Scheduler.	Malformed lines are skipped with a console warning so the application does not crash.
Which collections are used, and for what?	ArrayList for courses, rooms, instructors, and sessions.	The dataset size is moderate (around 30 courses and 15 rooms), making ArrayListefficient, easy to iterate, and simple to debug.

6. Automatic Scheduling Algorithm

6.1 Strategy

- **In what order are sessions processed?** Mandatory fixed-time sessions are processed first. Then, remaining sessions are scheduled course by course.
- **How are candidate time slots selected?** The algorithm loops through days (Monday to Friday) and hours (09:00 to 17:00), testing consecutive slots matching the session length.
- **How are suitable rooms selected?** The algorithm checks available rooms of the required type (Classroom or Laboratory) with capacity >= students.
- **How are scheduling constraints checked?** Before assigning, the system checks instructor availability (HC-01, HC-07), room availability (HC-02), and year-group overlap (HC-03).
- **How are conflicts handled?** If a room is busy, the algorithm tests the next room. If no room fits, it moves to the next time slot.
- **When does the algorithm decide a session cannot be assigned?** When all 40 slots have been evaluated and no valid combination satisfies all hard constraints.

6.2 Flowchart



☑ 1. Loading input data ☑ 2. Identifying fixed-time sessions ☑ 3. Selecting an unscheduled session ☑ 4. Searching for an available time slot ☑ 5. Searching for a suitable room ☑ 6. Checking scheduling constraints ☑ 7. Detecting conflicts ☑ 8. Trying alternatives ☑ 9. Recording successful assignments ☑ 10. Reporting unscheduled sessions ☑ 11. Completing the timetable generation process

6.3 Pseudocode

```
Load courses, instructors, rooms, and constraints from files
Identify and place fixed-time sessions; check hard constraints
For each unscheduled session in courses:
    placed = false
    For each day (Mon to Fri):
```

```
    For each starting hour (09:00 to 17:00 - session.hours):
      slot = TimeSlot(day, hour)
      If instructor is free AND year group is free:
        For each room in rooms:
          If room matches type AND capacity >= students AND room is free:
            Assign session to (slot, room)
            Check soft constraints (record warnings if any)
            placed = true
            Break from room and slot loops
    If placed:
      Break from day loop
  If not placed:
    Mark session as unscheduled (NO ROOM or CONSTRAINT)
Generate timetable report and show summary on console
```

6.4 Where Each Hard Constraint Is Checked

ID	Constraint	Checked in (Class.method)	Code version
HC-01	Instructor conflict	Timetable.isInstructorFree()	Planned for v1 / Done in v3
HC-02	Room conflict	Timetable.isRoomFree()	Planned for v1 / Done in v3
HC-03	Year-group conflict	Timetable.isYearGroupFree()	Planned for v1 / Done in v3
HC-04	Working hours	Scheduler.checkWorkingHours()	Planned for v1 / Done in v3
HC-05	Room capacity	Room.isSuitableFor()	Planned for v1 / Done in v2

HC-06	Room type	Room.isSuitableFor()	Planned for v1 / Done in v2
HC-07	Instructor availability	Instructor.isAvailable()	Planned for v1 / Done in v2
HC-08	Fixed-time requirements	Scheduler.scheduleFixedSessions()	Planned for v1 / Done in v3
HC-09	Room availability	Timetable.isRoomFree()	Planned for v1 / Done in v3

6.5 Known Limitations

Because the algorithm places sessions greedily into the first valid slot found, early placements might take up slots needed by later courses. This simple approach satisfies all hard constraints but may leave a course unscheduled even if a valid arrangement was theoretically possible with backtracking.

7. Conflict Detection and Handling

Problem	Detected in (Class.method)	What the algorithm does	What the user sees
Instructor or room already busy in slot	Timetable.isRoomFree(), isInstructorFree()	Tries the next room; if none work, tries the next time slot.	Screen 3 shows [OK] (after X rejected attempts) or failure.

Year-group overlap	Timetable.isYearGroupFree()	Skips the overlapping slot and tests the next available slot.	Prevents simultaneous mandatory classes for the same year.
No room with enough capacity or type	Room.isSuitableFor()	Skips unsuitable rooms; if no room exists, marks session as unscheduled.	Screen 5 lists session with NO ROOM tag for classroom request.
Fixed-time request violates a constraint	Scheduler.scheduleFixedSessions()	Leaves session unscheduled and logs the conflict.	Screen 3 and Screen 5 show [FAIL] CONSTRAINT problem.
Soft-constraint violation	Scheduler.checkSoftConstraints()	Accepts assignment and saves a warning.	Screen 3 and Screen 5 display [WARN] SC-xxwarning message.

8. User Interface Design

Menu option	What it does	Requirement IDs	Class.method called

1. Load input data	Reads CSV and TXT files and creates objects.	FR-01, FR-02, FR-03, FR-04, FR-05	Scheduler.loadData()
2. Generate timetable automatically	Runs the algorithm and assigns sessions to rooms and slots.	FR-07, FR-08, FR-09, FR-10	Scheduler.generateTimetable()
3. View timetable by year group	Displays a 5-day grid for the chosen year group.	FR-12	Timetable.printGrid(year)
4. View all scheduled sessions	Lists all placed sessions with times and rooms.	FR-13	Timetable.printAllScheduled()
5. View unscheduled sessions and warnings	Lists unassigned courses and soft-constraint warnings.	FR-11, FR-14	Scheduler.printUnscheduledAndWarnings()

6. Modify / remove an assignment	Manually changes or deletes an assigned session.	FR-15, FR-16, FR-17	Timetable.modifyAssignment()
7. Generate classroom request email	Creates a formal request draft for NO ROOM sessions.	FR-18, FR-19	Scheduler.generateClassroomRequest()
8. Save timetable to file	Exports the schedule to a text/CSV file.	FR-20	Timetable.saveToFile()
0. Exit	Terminates the application.	NFR-02	Main.main()

9. Requirements Traceability

Requirement	Implemented in (Class.method)	Test case(s)	Status
FR-01	Scheduler.loadCourses()	TC-01	Planned

FR-02	Scheduler.loadInstructors()	TC-02	Planned
FR-03	Scheduler.loadRooms()	TC-03	Planned
FR-04	Scheduler.loadConstraints()	TC-04	Planned
FR-07	Scheduler.scheduleFixedSessions()	TC-07	Planned
FR-08	Scheduler.scheduleRemainingSessions()	TC-08	Planned
HC-01	Timetable.isInstructorFree()	TC-09	Planned
HC-02	Timetable.isRoomFree()	TC-10	Planned
HC-03	Timetable.isYearGroupFree()	TC-11	Planned
HC-05	Room.isSuitableFor()	TC-05	Planned
HC-06	Room.isSuitableFor()	TC-06	Planned

10. Changes from the Previous Version

Empty

AI-Use Declaration

Tool	Used for	How we checked the result
ChatGPT / Gemini	We first drew the UML class diagram and scheduling flowchart sketches by hand on paper. We provided our hand-drawn layout and asked the AI to generate draw.io XML code for that exact structure. The AI also made minor corrections to layout alignment and formatting.	We imported the XML into draw.io, verified every class box, attribute, method, and multiplicity against Section 4 of the Project Guide, and exported the diagrams ourselves.
ChatGPT / Gemini	Clarifying project requirements from the assignment PDF and understanding the required Git folder structure (docs/, data/, output/, src/).	Cross-checked the file layout and submission checklist directly with Section 10 of the Project Guide and Tutorial Step 4.1.
ChatGPT / Gemini	Checking scheduling algorithm logic for potential flaws and conflict scenarios.	Tested sample course placements manually on paper using the project guide example data.
ChatGPT / Gemini	Learning Java language syntax, collection usage, and OOP best practices.	Compiled and tested sample snippets in IntelliJ IDEA to ensure complete understanding.

Grammarly / AI Assistant	Proofreading English grammar, spelling, and phrasing in project documentation.	Both team members read through the document to ensure clear and correct language.